

数据结构与算法八股文

一、线性数据结构

1. 数组和链表的区别？

对比项	数组	链表
内存布局	连续内存	离散内存（节点通过指针连接）
随机访问	$O(1)$ （通过下标）	$O(n)$ （需要遍历）
插入/删除	$O(n)$ （需要移动元素）	$O(1)$ （修改指针，但找位置需 $O(n)$ ）
内存分配	静态/动态，需预先指定大小	动态分配，按需扩展
缓存友好性	高（连续内存，预取效果好）	低（节点分散，缓存命中率低）
适用场景	频繁随机访问，数据量固定	频繁插入删除，数据量动态变化

延伸：数组和链表的结合体 —— **跳表（Skip List）**
跳表在链表基础上增加多级索引，将查找时间复杂度从 $O(n)$ 降到 $O(\log n)$ ，Redis 的有序集合（ZSet）底层就是跳表。

2. 如何反转链表？

迭代法（推荐， $O(1)$ 空间）：

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(nullptr) {}
};

ListNode* reverseList(ListNode *head) {
    ListNode *prev = nullptr;
    ListNode *curr = head;
    while (curr) {
        ListNode *next = curr->next; // 保存下一个节点
        curr->next = prev;           // 反转指针
        prev = curr;                 // prev 前进
        curr = next;                 // curr 前进
    }
    return prev;
}
```

递归法：

```

ListNode* reverseListRecursive(ListNode *head) {
    if (!head || !head->next) return head;
    ListNode *newHead = reverseListRecursive(head->next);
    head->next->next = head; // 反转
    head->next = nullptr; // 防止环
    return newHead;
}

```

复杂度：时间 $O(n)$ ，空间 $O(1)$ （迭代） / $O(n)$ （递归栈）。

3. 如何判断链表是否有环？如何找到环的入口？

快慢指针法（Floyd 判圈算法）：

```

ListNode *detectCycle(ListNode *head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) { // 相遇，有环
            // 找入口：一个指针从头开始，一个从相遇点开始，同步走
            ListNode *ptr = head;
            while (ptr != slow) {
                ptr = ptr->next;
                slow = slow->next;
            }
            return ptr; // 环入口
        }
    }
    return nullptr;
}

```

原理：

- 快指针每次走 2 步，慢指针每次走 1 步。如果有环，快指针一定会追上慢指针。
- 设头到入口距离为 a ，入口到相遇点距离为 b ，相遇点到入口距离为 c 。
- 相遇时：慢指针走了 $a + b$ ，快指针走了 $a + b + k(b + c)$ 。
- 因为快指针速度是慢指针 2 倍： $2(a + b) = a + b + k(b + c) \rightarrow a = c + (k-1)(b+c)$ 。
- 所以从头和从相遇点同步走，一定会在入口相遇。

4. 栈和队列的区别及相互实现？

区别：

- 栈 (Stack)：后进先出 (LIFO)，操作在栈顶。
- 队列 (Queue)：先进先出 (FIFO)，操作在队头和队尾。

用两个栈实现队列：

```

class MyQueue {

```

```

    stack<int> inStack, outStack;
public:
    void push(int x) { inStack.push(x); }

    int pop() {
        if (outStack.empty()) {
            while (!inStack.empty()) {
                outStack.push(inStack.top());
                inStack.pop();
            }
        }
        int val = outStack.top();
        outStack.pop();
        return val;
    }

    int peek() {
        if (outStack.empty()) {
            while (!inStack.empty()) {
                outStack.push(inStack.top());
                inStack.pop();
            }
        }
        return outStack.top();
    }

    bool empty() { return inStack.empty() && outStack.empty(); }
};

```

复杂度：push $O(1)$, pop 均摊 $O(1)$ 。

用两个队列实现栈：

```

class MyStack {
    queue<int> q1, q2;
public:
    void push(int x) {
        q2.push(x);
        while (!q1.empty()) {
            q2.push(q1.front());
            q1.pop();
        }
        swap(q1, q2);
    }

    int pop() {
        int val = q1.front();
        q1.pop();
        return val;
    }

    int top() { return q1.front(); }
    bool empty() { return q1.empty(); }
};

```

5. 最小栈如何实现？

设计一个栈，支持 push、pop、top，并在 O(1) 时间内检索到最小元素。

```
class MinStack {
    stack<int> dataStack;
    stack<int> minStack; // 辅助栈，记录当前最小值
public:
    void push(int val) {
        dataStack.push(val);
        if (minStack.empty() || val <= minStack.top())
            minStack.push(val);
    }

    void pop() {
        if (dataStack.top() == minStack.top())
            minStack.pop();
        dataStack.pop();
    }

    int top() { return dataStack.top(); }
    int getMin() { return minStack.top(); }
};
```

复杂度：所有操作均为 O(1)，空间 O(n)。

二、哈希表

6. 哈希表的底层原理和冲突解决？

哈希表：通过哈希函数将键映射到数组索引，实现 O(1) 平均查找。

冲突解决：

1. **链地址法 (Separate Chaining)：**每个桶是一个链表，冲突时追加到链表。Java HashMap、C++ `unordered_map` 使用此方法。
2. **开放地址法 (Open Addressing)：**冲突时按探测序列找下一个空位。
 - 线性探测： $h(k) + i$
 - 二次探测： $h(k) + i^2$
 - 双重哈希： $h1(k) + i * h2(k)$

C++ `unordered_map` 底层：

- 底层是哈希桶数组，每个桶是链表（链地址法）。
- 负载因子 (load_factor) = 元素数 / 桶数。当负载因子超过阈值（通常 1.0）时，触发 rehash（扩容并重新散列）。

延伸：HashMap 为什么线程不安全？

- 多线程同时 put 可能导致数据覆盖。
- Java 7 中扩容时可能形成环形链表，导致死循环（Java 8 已修复）。
- 线程安全替代：`ConcurrentHashMap`（分段锁/CAS）、C++ 中需加锁。

7. 如何设计一个 LRU 缓存?

LRU (Least Recently Used) : 最近最少使用淘汰策略。

实现: 哈希表 + 双向链表。

- 哈希表: $O(1)$ 查找节点。
- 双向链表: $O(1)$ 插入、删除、移动节点到头部。

```
struct Node {
    int key, val;
    Node *prev, *next;
    Node(int k, int v) : key(k), val(v), prev(nullptr), next(nullptr) {}
};

class LRUCache {
    int capacity;
    unordered_map<int, Node*> map;
    Node *head, *tail; // 虚拟头尾节点

    void addToHead(Node *node) {
        node->next = head->next;
        node->prev = head;
        head->next->prev = node;
        head->next = node;
    }

    void removeNode(Node *node) {
        node->prev->next = node->next;
        node->next->prev = node->prev;
    }

    void moveToHead(Node *node) {
        removeNode(node);
        addToHead(node);
    }

    Node* removeTail() {
        Node *node = tail->prev;
        removeNode(node);
        return node;
    }

public:
    LRUCache(int cap) : capacity(cap) {
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head->next = tail;
        tail->prev = head;
    }

    int get(int key) {
        if (map.find(key) == map.end()) return -1;
        Node *node = map[key];
```

```

        moveToHead(node);
        return node->val;
    }

    void put(int key, int value) {
        if (map.find(key) != map.end()) {
            Node *node = map[key];
            node->val = value;
            moveToHead(node);
        } else {
            if (map.size() >= capacity) {
                Node *removed = removeTail();
                map.erase(removed->key);
                delete removed;
            }
            Node *newNode = new Node(key, value);
            map[key] = newNode;
            addToHead(newNode);
        }
    }
};

```

复杂度: get/put 均为 $O(1)$ 。

三、树与二叉树

8. 二叉树的遍历方式?

递归遍历:

```

struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// 前序: 根-左-右
void preorder(TreeNode *root, vector<int> &res) {
    if (!root) return;
    res.push_back(root->val);
    preorder(root->left, res);
    preorder(root->right, res);
}

// 中序: 左-根-右 (BST 中序遍历有序)
void inorder(TreeNode *root, vector<int> &res) {
    if (!root) return;
    inorder(root->left, res);
    res.push_back(root->val);
    inorder(root->right, res);
}

// 后序: 左-右-根
void postorder(TreeNode *root, vector<int> &res) {

```

```

    if (!root) return;
    postorder(root->left, res);
    postorder(root->right, res);
    res.push_back(root->val);
}

```

层序遍历 (BFS) :

```

vector<vector<int>> levelorder(TreeNode *root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int size = q.size();
        vector<int> level;
        for (int i = 0; i < size; i++) {
            TreeNode *node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

9. 二叉搜索树 (BST) 的性质和操作?

性质:

- 左子树所有节点值 < 根节点值
- 右子树所有节点值 > 根节点值
- 左右子树也是 BST
- **中序遍历 BST 得到有序序列**

查找:

```

TreeNode* searchBST(TreeNode *root, int val) {
    if (!root || root->val == val) return root;
    if (val < root->val) return searchBST(root->left, val);
    return searchBST(root->right, val);
}

```

插入:

```

TreeNode* insertBST(TreeNode *root, int val) {
    if (!root) return new TreeNode(val);
    if (val < root->val) root->left = insertBST(root->left, val);
    else root->right = insertBST(root->right, val);
    return root;
}

```

删除（三种情况）：

1. 叶子节点：直接删除。
2. 只有一个子节点：用子节点替代。
3. 有两个子节点：找右子树最小节点（或左子树最大节点）替代，然后删除该节点。

10. 如何判断二叉树是否平衡？

平衡二叉树 (AVL)： 任意节点的左右子树高度差不超过 1。

```
int height(TreeNode *root) {
    if (!root) return 0;
    int leftH = height(root->left);
    if (leftH == -1) return -1;
    int rightH = height(root->right);
    if (rightH == -1) return -1;
    return abs(leftH - rightH) > 1 ? -1 : max(leftH, rightH) + 1;
}

bool isBalanced(TreeNode *root) {
    return height(root) != -1;
}
```

复杂度： 时间 $O(n)$ ，每个节点只访问一次。

四、图

11. 图的存储方式？

1. **邻接矩阵：** 二维数组 `adj[i][j]` 表示边。适合稠密图，空间 $O(V^2)$ 。
2. **邻接表：** 数组 + 链表（或 vector）。适合稀疏图，空间 $O(V + E)$ 。

```
// 邻接表实现
class Graph {
    int v;
    vector<vector<int>>> adj;
public:
    Graph(int v) : v(v), adj(v) {}
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u); // 无向图
    }
};
```

12. 图的遍历（BFS 和 DFS）？

BFS（广度优先）：用队列，适合求最短路径（无权图）。

```
void bfs(Graph &g, int start) {
    vector<bool> visited(g.V, false);
    queue<int> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for (int v : g.adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

DFS（深度优先）：用递归或栈，适合拓扑排序、连通分量。

```
void dfs(Graph &g, int u, vector<bool> &visited) {
    visited[u] = true;
    cout << u << " ";
    for (int v : g.adj[u]) {
        if (!visited[v]) dfs(g, v, visited);
    }
}
```

复杂度：时间 $O(V + E)$ ，空间 $O(V)$ 。

13. 最短路径算法有哪些？

算法	适用场景	时间复杂度	特点
BFS	无权图单源最短	$O(V + E)$	层序遍历
Dijkstra	非负权图单源最短	$O((V+E)\log V)$	贪心，优先队列
Bellman-Ford	含负权图单源最短	$O(VE)$	可检测负环
Floyd	多源最短路径	$O(V^3)$	动态规划

Dijkstra 实现（优先队列优化）：

```
vector<int> dijkstra(Graph &g, int src) {
    vector<int> dist(g.V, INT_MAX);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
```

```

    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (auto &[v, w] : g.adj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            pq.push({dist[v], v});
        }
    }
}
return dist;
}

```

五、排序算法

14. 常见排序算法的复杂度对比？

算法	平均时间	最好时间	最坏时间	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定

面试常考：快排、归并、堆排。

15. 快速排序的实现和原理？

原理：分治法。选一个基准（pivot），将数组分为小于和大于基准的两部分，递归排序。

```

int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[high]; // 选最后一个元素为基准
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int> &arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
    }
}

```

```

        quickSort(arr, pi + 1, high);
    }
}

```

优化:

- **三数取中**: 选 low、mid、high 的中位数作为 pivot，避免最坏情况。
- **小数组用插入排序**: 当子数组长度 < 15 时，切换为插入排序。
- **尾递归优化**: 先递归较小部分，避免栈溢出。

16. 归并排序的实现?

原理: 分治法。将数组分成两半，分别排序，然后合并。

```

void merge(vector<int> &arr, int left, int mid, int right) {
    vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) temp[k++] = arr[i++];
        else temp[k++] = arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    for (int x = 0; x < k; x++) arr[left + x] = temp[x];
}

void mergeSort(vector<int> &arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

```

应用: 求逆序对数量 (归并时统计)。

17. 堆排序的实现?

原理: 利用大顶堆 (或小顶堆) 的性质。先建堆，然后依次将堆顶元素与末尾交换，调整堆。

```

void heapify(vector<int> &arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1, right = 2 * i + 2;
    if (left < n && arr[left] > arr[largest]) largest = left;
    if (right < n && arr[right] > arr[largest]) largest = right;
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int> &arr) {

```

```

int n = arr.size();
// 建堆（从最后一个非叶子节点开始）
for (int i = n / 2 - 1; i >= 0; i--)
    heapify(arr, n, i);
// 排序
for (int i = n - 1; i > 0; i--) {
    swap(arr[0], arr[i]);
    heapify(arr, i, 0);
}
}

```

应用：Top K 问题（用小顶堆维护 K 个最大元素）。

六、查找算法

18. 二分查找的实现和注意事项？

前提：数组有序。

```

// 标准二分查找
int binarySearch(vector<int> &arr, int target) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2; // 防止溢出
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

// 查找第一个等于 target 的位置（左边界）
int lowerBound(vector<int> &arr, int target) {
    int left = 0, right = arr.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] < target) left = mid + 1;
        else right = mid;
    }
    return left;
}

// 查找最后一个等于 target 的位置（右边界）
int upperBound(vector<int> &arr, int target) {
    int left = 0, right = arr.size();
    while (left < right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] <= target) left = mid + 1;
        else right = mid;
    }
    return left - 1;
}

```

注意事项:

- `mid = left + (right - left) / 2` 防止 `(left + right)` 溢出。
- 循环条件 `left <= right` (闭区间) 或 `left < right` (左闭右开)。
- 更新边界时注意是否包含 `mid`。

七、动态规划

19. 动态规划的核心思想?

核心: 将大问题分解为重叠子问题, 通过保存子问题的解 (备忘录/DP 表) 避免重复计算。

步骤:

1. 定义状态 (dp 数组的含义)
2. 确定状态转移方程
3. 确定初始条件 (base case)
4. 确定遍历顺序

20. 0-1 背包问题?

问题: n 个物品, 每个有重量 w 和价值 v , 背包容量 W , 求最大价值。

```
int knapsack(vector<int> &w, vector<int> &v, int W) {
    int n = w.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp[i][j] = dp[i-1][j]; // 不选
            if (j >= w[i-1])
                dp[i][j] = max(dp[i][j], dp[i-1][j-w[i-1]] + v[i-1]); // 选
        }
    }
    return dp[n][W];
}
```

空间优化 (滚动数组):

```
int knapsackOpt(vector<int> &w, vector<int> &v, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < w.size(); i++) {
        for (int j = W; j >= w[i]; j--) // 逆序遍历, 防止重复使用
            dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
    }
    return dp[W];
}
```

21. 最长递增子序列 (LIS) ?

```
int lengthOfLIS(vector<int> &nums) {
    vector<int> dp; // dp[i] 表示长度为 i+1 的 LIS 的最小末尾元素
    for (int num : nums) {
        auto it = lower_bound(dp.begin(), dp.end(), num);
        if (it == dp.end()) dp.push_back(num);
        else *it = num;
    }
    return dp.size();
}
```

复杂度：时间 $O(n \log n)$ ，空间 $O(n)$ 。

八、贪心算法

22. 贪心算法与动态规划的区别？

对比项	贪心算法	动态规划
核心思想	每一步选局部最优，期望得到全局最优	考虑所有子问题，选全局最优
适用条件	贪心选择性质 + 最优子结构	最优子结构 + 重叠子问题
正确性	需要证明贪心策略正确	只要状态转移正确就正确
效率	通常更高 ($O(n)$ 或 $O(n \log n)$)	通常较低 ($O(n^2)$ 或更高)

经典贪心问题：

- 活动选择问题（按结束时间排序）
- 霍夫曼编码（优先队列）
- 最小生成树（Kruskal、Prim）
- 找零钱问题（特定面额下）

23. 跳跃游戏 (贪心) ?

```
bool canJump(vector<int> &nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (i > maxReach) return false; // 无法到达当前位置
        maxReach = max(maxReach, i + nums[i]);
        if (maxReach >= nums.size() - 1) return true;
    }
    return true;
}
```

思路：维护当前能到达的最远位置，遍历更新。

九、综合面试题

24. 如何判断一个数是否是 2 的幂？

```
bool isPowerOfTwo(int n) {  
    return n > 0 && (n & (n - 1)) == 0;  
}
```

原理：2 的幂二进制只有一个 1，`n & (n-1)` 会消除最低位的 1。

25. 如何求两个数的最大公约数？

```
int gcd(int a, int b) {  
    while (b) {  
        a %= b;  
        swap(a, b);  
    }  
    return a;  
}
```

原理：辗转相除法（欧几里得算法）。

26. 如何判断括号是否有效？

```
bool isValid(string s) {  
    stack<char> st;  
    for (char c : s) {  
        if (c == '(' || c == '[' || c == '{') st.push(c);  
        else {  
            if (st.empty()) return false;  
            char top = st.top(); st.pop();  
            if (c == ')' && top != '(') return false;  
            if (c == ']' && top != '[') return false;  
            if (c == '}' && top != '{') return false;  
        }  
    }  
    return st.empty();  
}
```

27. 如何合并两个有序链表?

```
ListNode* mergeTwoLists(ListNode *l1, ListNode *l2) {
    ListNode dummy(0);
    ListNode *curr = &dummy;
    while (l1 && l2) {
        if (l1->val < l2->val) { curr->next = l1; l1 = l1->next; }
        else { curr->next = l2; l2 = l2->next; }
        curr = curr->next;
    }
    curr->next = l1 ? l1 : l2;
    return dummy.next;
}
```

28. 如何求二叉树的最大深度?

```
int maxDepth(TreeNode *root) {
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

29. 如何验证二叉搜索树?

```
bool isValidBST(TreeNode *root, long minVal = LONG_MIN, long maxVal = LONG_MAX) {
    if (!root) return true;
    if (root->val <= minVal || root->val >= maxVal) return false;
    return isValidBST(root->left, minVal, root->val) &&
        isValidBST(root->right, root->val, maxVal);
}
```

30. 如何求数组中的第 K 大元素?

方法1: 快速选择 (Quick Select) , 平均 $O(n)$ 。

```
int findKthLargest(vector<int> &nums, int k) {
    int target = nums.size() - k;
    int left = 0, right = nums.size() - 1;
    while (left < right) {
        int pivot = nums[right];
        int i = left;
        for (int j = left; j < right; j++) {
            if (nums[j] < pivot) swap(nums[i++], nums[j]);
        }
        swap(nums[i], nums[right]);
        if (i == target) return nums[i];
        else if (i < target) left = i + 1;
        else right = i - 1;
    }
    return nums[left];
}
```



```
}

```

方法2: 小顶堆, $O(n \log k)$ 。

```
int findKthLargestHeap(vector<int> &nums, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int num : nums) {
        pq.push(num);
        if (pq.size() > k) pq.pop();
    }
    return pq.top();
}
```

附录：算法复杂度速查表

复杂度	典型算法/操作
$O(1)$	哈希表查找、数组随机访问
$O(\log n)$	二分查找、平衡树操作
$O(n)$	线性遍历、BFS/DFS
$O(n \log n)$	快排、归并、堆排
$O(n^2)$	冒泡、选择、插入排序
$O(2^n)$	递归求斐波那契（无备忘录）
$O(n!)$	全排列

面试高频算法题分类

分类	高频题目
链表	反转链表、合并链表、环检测、LRU 缓存
树	遍历、最大深度、验证 BST、最近公共祖先
排序	快排、归并、Top K
二分	搜索旋转数组、寻找峰值
动态规划	背包、LIS、最长公共子序列、编辑距离
贪心	跳跃游戏、活动选择
回溯	全排列、N 皇后、组合总和
图	BFS/DFS、最短路径、拓扑排序